Laboratory 1

Víctor Brull Rico

April 2026

1 Introduction

Feature detection is a fundamental step in many computer vision tasks, such as image matching, object recognition, 3D reconstruction from multiple views, stereo correspondence, and motion tracking [1]. The core idea is to identify a set of distinctive locations in an image, called *keypoints*, that can be reliably detected and matched across different views of the same scene, even under changes in scale, rotation, illumination, or viewpoint.

One of the most influential methods for feature detection is the Scale Invariant Feature Transform (SIFT), introduced by Lowe [1]. SIFT detects keypoints by searching for stable extrema across all scales of an image using a *scale space* representation. The scale space is built by convolving the image with Gaussian filters at increasing values of σ, and keypoint candidates are identified as local maxima or minima of the Difference of Gaussians (DoG) function, which serves as an efficient approximation to the scale-normalized Laplacian of Gaussian [1]. A point is selected as a keypoint only if it is an extremum with respect to its neighbors in the current and adjacent scales of the DoG pyramid. This design makes the detected keypoints invariant to image scale and rotation, and partially invariant to changes in illumination and 3D camera viewpoint [1].

The goal of this laboratory is to gain practical experience with feature detection through two complementary tasks. In the first part, we implement the SIFT scale-space extrema detection algorithm in MATLAB. Specifically, we study the provided functions that build the scale space and compute the Difference of Gaussians, and we implement the missing extrema detection logic in `find_extremas.m`. Additionally, we test our implementation on images from different scenarios, including indoor and outdoor scenes as well as textureless regions, and discuss the quality of the detected keypoints. In the second part, we compare several feature detection algorithms available in MATLAB, such as SIFT, SURF, ORB, FAST, KAZE, and HARRIS. For each detector, we analyze the effect of its parameters on the image `sunflower.jpg` and provide a quantitative comparison in terms of number of keypoints, computation time, type of features detected, and behavior in challenging scenarios such as textureless areas.

2 Methods

This section details the implementation of the SIFT scale-space extrema detector, describing the construction of the Gaussian pyramid, the computation of the Difference of Gaussians (DoG), and the extrema detection code implemented in `find_extremas.m`.

Scale-space construction

The function `scale_space.m` builds a multi-octave Gaussian pyramid from the input image by repeatedly smoothing with Gaussian kernels of increasing standard deviation σ and by changing image resolution across octaves. In practice the procedure implemented is:

- For a given octave, generate a sequence of blurred images by convolving the image with Gaussian kernels of increasing σ using `gaussian_filter.m`.
- After computing enough scale levels for the current octave, downsample (using `half_image.m`) to produce the base image for the next octave; conversely, when `params.omin` is negative the code first upsamples using `double_image.m`.

The result is a cell array `pyr{o}` for octave index o, where each `pyr{o}(:,:,s)` corresponds to the image at scale index s within that octave. The multiplicative scale step between adjacent levels is $k = 2^{1/S}$ (where `S = params.S`), so scales are spaced geometrically. A compact schematic of this pipeline is shown below.

$$\begin{gathered}\text{Original image}\\ \downarrow\\ \text{Octave 1: } I * G_{\sigma_1} \to I * G_{\sigma_2} \to \cdots\\ \downarrow \text{ (downsample)}\\ \text{Octave 2: } (I \downarrow) * G_{\sigma_1} \to \cdots\end{gathered}$$

Figure 1: Schematic representation of Gaussian pyramid construction: each octave contains several increasingly blurred images, and octaves are linked by resampling.

Difference of Gaussians (DoG)

The file `difference_of_gaussians.m` computes, for each octave, the difference between consecutive blurred images in the corresponding octave:

$$\mathrm{DoG}_o(x, y, s) = \mathrm{G}_o(x, y, s + 1) - \mathrm{G}_o(x, y, s),$$

where $\mathrm{G}_o(:, :, s)$ denotes the Gaussian-blurred image at octave o and scale index s. This difference approximates the Laplacian of Gaussian (LoG) response, which is sensitive to blob-like structures across scales. The DoG representation is both efficient to compute and well suited to finding scale-invariant keypoints because extrema of the LoG across scale correspond to characteristic scales of image structures.

Extrema detection: implementation in `find_extremas.m`

The core task implemented in `find_extremas.m` is to identify the local extrema of the DoG across both space and scale. The chosen algorithm follows Lowe's SIFT description: for each sample location in the DoG volume (excluding border pixels and the first/last scale), compare the current value to its neighbors. A point is accepted as a keypoint if it is strictly greater than all the neighbors (a local maximum) or strictly smaller than all the neighbors (a local minimum), and its absolute response exceeds a magnitude threshold `params.thresh`.

Key implementation details and design choices:

- **Neighborhood extraction:** for a given octave `o`, scale `s` and pixel `(y,x)` the code extracts a 3×3×3 block `nb = dog{o}(y-1:y+1, x-1:x+1, s-1:s+1)` and flattens it (in the case of radius 2); the center element is removed for comparison.
- **Strict comparison:** maxima are accepted only if `val > all(neighbors)` and minima only if `val < all(neighbors)`; this avoids ambiguity when equal values appear.
- **Magnitude thresholding:** `abs(val) > params.thresh` is required to filter weak responses and reduce unstable detections.
- **Mapping to the original image:** keypoints detected at octave `o` and spatial coordinates `(y,x)` are mapped back to the coordinate system of the original input by scaling with the octave factor: `ypt = round(y * 2^(params.omin + o - 1))`, `xpt = round(x * 2^(params.omin + o - 1))`. Coordinates are clamped to image bounds before writing into the global binary `keypoints` image.
- **Boundary handling:** the search excludes the 1-pixel borders of each DoG slice and does not attempt sub-pixel localization; this keeps the implementation simple while remaining faithful to the essential SIFT detection step.

The pseudocode below summarizes the main loop:

for $o = 1$ to `params.O` **do**
  $[\mathtt{M}, \mathtt{N}, \mathtt{S}] = \mathrm{size}(\mathtt{dog\{o\}})$

```
    for s = 2 to S − 1 do
      for y = 2 to M − 1 do
        for x = 2 to N − 1 do
          val = dog{o}(y,x,s)
          nb = dog{o}(y-1:y+1, x-1:x+1, s-1:s+1)
          remove center from nb
          if val > all(nb) and |val| > params.thresh then
            mark keypoint (maximum)
          else if val < all(nb) and |val| > params.thresh then
            mark keypoint (minimum)
          end if
        end for
      end for
    end for
  end for
```

This implementation produces a binary image `keypoints` of the same size as the original input, where detected keypoint locations are set to 1. The result is intentionally minimal (no orientation assignment, no sub-pixel refinement), focusing on correct identification of scale-space extrema.

3 Experiments

This section presents the results of our SIFT keypoint detector implementation. For each test image we show three panels side by side: the original image, the keypoint overlay (`-kp`), and the binary keypoint map (`-bin`).

Datasets used

We tested the detector on four images covering three different scenarios:

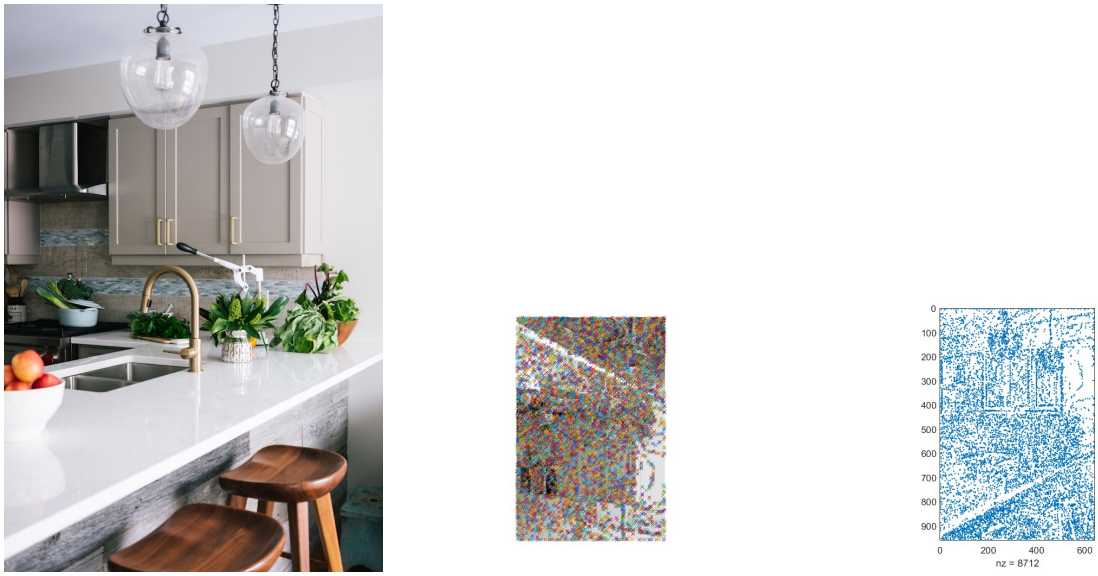
- **Indoor scene:** `Figures/indoor.jpg`
- **Outdoor scene:** `Figures/landscape-a.jpg`
- **Textureless scenes:** `Figures/textureless-a.jpg` and `Figures/textureless-b.jpg`

Results per scenario

Indoor The indoor image produces a large number of keypoints, mostly concentrated around high-contrast structures such as edges, corners, and textured surfaces like furniture or walls with patterns. Flat, uniform regions such as plain walls or floors produce very few detections, which is expected since the DoG response is weak where gradients are small [1]. The binary map confirms that detections are well localized around structural elements.

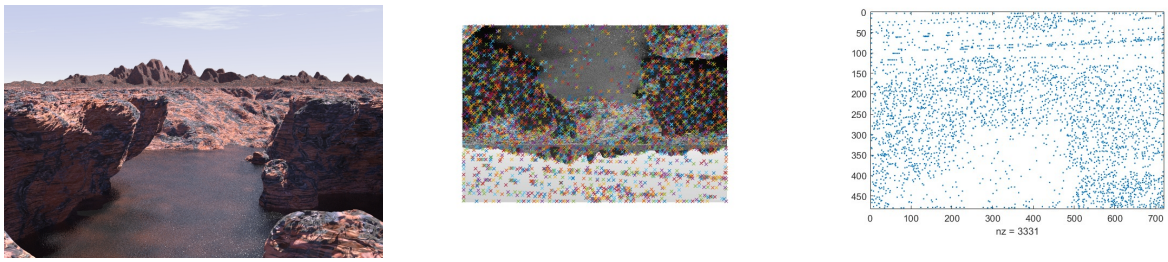
Outdoor The outdoor image shows a rocky landscape with cliffs and water. As seen in Fig. 3, keypoints are densely concentrated in the rocky surfaces, where the rough texture generates strong and varied gradients at multiple scales. In contrast, the sky region at the top and the relatively smooth water surface produce very few detections, since those areas lack the local gradient structure needed to generate DoG extrema. This behaviour is consistent with what was observed indoors when the detector responds to structure and texture, and ignores uniform regions regardless of the scene type.

Textureless scenes As expected, images with little texture produce very few keypoints. In some cases the detector still picks up spurious responses caused by noise, JPEG compression artefacts, or specular highlights. These are false positives: the DoG response at those locations is weak and unstable, meaning the detected points would not be reliable for matching [1]. Raising `params.thresh` helps suppress most of these detections.



Original | Keypoint overlay | Binary map

Figure 2: Indoor scene: original image (left), detected keypoints overlaid (centre), and binary keypoint map (right).



Original | Keypoint overlay | Binary map

Figure 3: Outdoor scene: original image (left), detected keypoints overlaid (centre), and binary keypoint map (right).

Outlier analysis

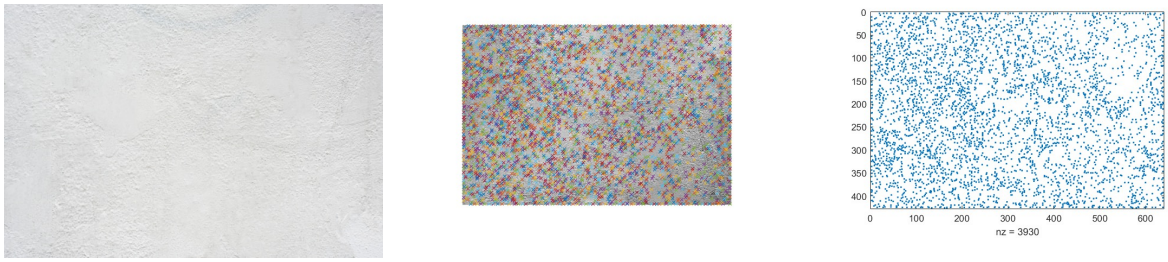
Some detected keypoints are not reliable and should be considered outliers. The main cases we observed are:

- **Specular highlights and bright spots:** isolated high-contrast points in otherwise smooth regions that are not useful for matching.
- **Edge responses:** points lying along long edges pass the extrema test but are poorly localized along the edge direction, making them unstable [1].
- **Noise and compression artefacts:** in low-texture images, small pixel-level variations can generate spurious local extrema.

To reduce these outliers, three straightforward strategies can be applied. First, **low-contrast thresholding** consists of increasing `params.thresh` to discard weak DoG responses, which is the simplest fix and is especially effective for textureless images. Lowe suggests discarding all extrema with a DoG value below 0.03 [1]. Second, **edge response rejection** uses the Hessian matrix at each candidate point to compute the ratio of principal curvatures and reject points that lie on edges rather than blobs; Lowe recommends a threshold of $r = 10$ on this ratio [1]. Third, **non-maximum suppression** enforces a minimum distance between keypoints to avoid dense clusters and produce a more uniform spatial distribution across the image.

Original | Keypoint overlay | Binary map

Figure 4: Textureless scene A: original image (left), detected keypoints overlaid (centre), and binary keypoint map (right).



Original | Keypoint overlay | Binary map

Figure 5: Textureless scene B: original image (left), detected keypoints overlaid (centre), and binary keypoint map (right).

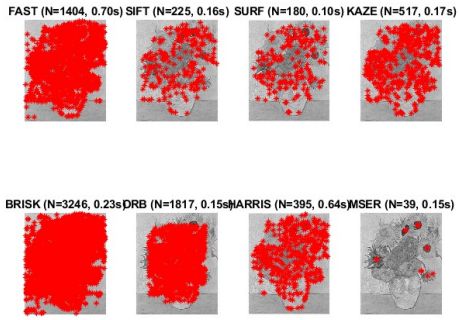
Feature detection algorithm comparison

In this section we compare seven feature detectors available in MATLAB on the image `sunflower.jpg`, shown together with the detection results in Fig. 6. For each detector we report the number of detected keypoints, computation time, and qualitative behaviour.

Parameter tuning For each detector we explored the main parameters and their effect on the number and distribution of detected keypoints. For **SIFT**, lowering `ContrastThreshold` increases detections (including weak, noisy ones), while increasing `EdgeThreshold` keeps more edge-like responses. `NumOctaveLayers` controls scale sampling density and `Sigma` sets the initial blur level. For **SURF**, a higher `MetricThreshold` keeps only strong blob responses and reduces the total count. For **FAST** and **BRISK**, the contrast threshold directly controls corner sensitivity: lowering it increases detections but also picks up noise. For **KAZE**, reducing `Threshold` yields more features and increasing `NumOctaves` or `NumScaleLevels` improves multi-scale coverage at the cost of computation time. For **ORB**, a smaller `ScaleFactor` produces more pyramid levels and thus more detections. Finally, for **HARRIS** and **MSER**, `MinQuality` and `RegionAreaRange` respectively control the strictness of the detection criteria.

Comparison table Table 1 summarises the results obtained with the tuned parameters for each algorithm on `sunflower.jpg`.

Original `sunflower.jpg`



Detector comparison

Figure 6: Left: original `sunflower.jpg`. Right: keypoint detections for all tested algorithms, with the number of keypoints and computation time shown per detector.

Table 1: Feature detector comparison on `sunflower.jpg`. Scale invariance and textureless behaviour are assessed qualitatively.

Algorithm	Type	# Keypoints	Time (s)	Scale invariant	Textureless areas	Notes
FAST	Corner	1404	0.70	No	Poor	Very fast; many detections but no scale or orientation info
SIFT	Blob	225	0.16	Yes	Poor	Distinctive and stable; good for accurate matching
SURF	Blob	180	0.10	Yes	Poor	Faster approximation of SIFT; fewer but strong detections
KAZE	Blob	517	0.17	Yes	Poor	Nonlinear scale space; good localization in structured regions
BRISK	Corner	3246	0.23	Partial	Poor	Large number of detections; suitable for real-time use
ORB	Corner	1817	0.15	Partial	Poor	Binary descriptor; efficient and rotation-aware
HARRIS	Corner	395	0.64	No	Poor	Classic corner detector; slow and not scale-invariant
MSER	Region	39	0.15	Yes	Very poor	Detects stable regions; very few detections on this image

Discussion The results in Table 1 reveal clear trade-offs between the algorithms. In terms of speed, SURF is the fastest (0.10 s) followed closely by ORB, KAZE, MSER, and SIFT, while FAST and HARRIS are slower despite being conceptually simpler. Regarding keypoint quantity, BRISK detects by far the most points (3246), followed by ORB and FAST, whereas MSER produces the fewest (39). When it comes to scale invariance, SIFT, SURF, and KAZE are fully scale-invariant by design, making them more suitable for matching across different image scales, while FAST and HARRIS lack this property entirely. All detectors struggle in textureless areas, confirming that local feature detectors inherently depend on gradient structure to produce reliable responses. Overall, SIFT and KAZE offer the best balance between distinctiveness and scale invariance, while BRISK and ORB are preferable when speed and a high number of detections are the priority.

4 Conclusion

Implementing SIFT from scratch was instructive in understanding how scale-space extrema detection works in practice. The key insight is that the DoG pyramid efficiently approximates the Laplacian of Gaussian across scales, and that stable keypoints are those that simultaneously stand out from their spatial and scale neighbors. Building the detector step by step, from Gaussian blurring to all the neighbors comparison, made clear why the method is robust to scale changes but also why it is sensitive to noise and thresholding choices, as seen in the textureless results of Figs. 4 and 5.

Regarding detector choice, the best option depends on the application. SIFT and KAZE are the most reliable for accuracy-critical tasks such as image matching or 3D reconstruction, since they are

scale-invariant and produce distinctive, well-localized keypoints. By contrast, BRISK and ORB are better suited to real-time applications where speed and a large number of detections matter more than precision, as confirmed by the timing results in Table 1.

The main limitation shared by all tested detectors is their poor performance in textureless regions, where the absence of gradient structure leads to very few or unreliable detections. Beyond that, some detectors (FAST, HARRIS) lack scale invariance, which limits their applicability when images are captured at different resolutions or distances. Finally, most detectors are sensitive to parameter tuning: a poorly chosen threshold can either miss true keypoints or produce many spurious ones, which highlights the importance of careful calibration for each specific use case.

References

[1] D. G. Lowe, "Distinctive image features from scale-invariant keypoints," *International Journal of Computer Vision*, vol. 60, no. 2, pp. 91–110, 2004.